

[University Name]
[Faculty / Department] · [Degree Programme]

Robot Operating System 2: Architecture, Communication and Application in a Manipulator Robot

Assignment report

Student: [Student Name] ([Student ID])

Module: [Module Name and Code]

Instructor / supervisor: [Supervisor Name]

Course studied: *Robotics and ROS 2 – Learn by Doing! Manipulators* (course material and repository by Antonio Brandi)

Target environment: Ubuntu 22.04 LTS, ROS 2 Humble Hawksbill

Submission date: [Submission Date]

Placeholders in square brackets are to be completed by the student.

Abstract

This report explains the Robot Operating System 2 (ROS 2) through one concrete project: the three-degree-of-freedom manipulator with gripper ("arduinoobot") of the course *Robotics and ROS 2 – Learn by Doing! Manipulators*. Three kinds of source were used and kept apart: the student's own course notes, which establish what was studied and practised; the instructor's public repository, whose Section 4 workspace at commit 4936385 was taken as the software baseline; and the official ROS 2 Humble documentation, which supports the technical explanations.

A clean workspace with three packages was prepared and twelve small, documented corrections were made, concerning the timer period, clean shutdown, parameter validation, dependency declarations and a stale RViz configuration. The unchanged robot model and the corrected examples were built and exercised in an Ubuntu 22.04 / ROS 2 Humble container: Python-to-C++ and C++-to-Python publisher-subscriber communication, parameter inspection and updates including the rejection of invalid updates, URDF processing, joint states, TF and an RViz display rendered on a virtual screen. The notes document the installation, workspace creation, publishers and subscribers in both languages and a complete URDF model visualised in RViz; the parameter laboratory is documented only as a code listing.

The main limitations are that verification ran in a container rather than on the student's own machine, that RViz used software rendering, that simulation, `ros2_control`, MoveIt 2 and the physical robot were not exercised, and that the robot description is a kinematic model at ten times the physical size with placeholder dynamic parameters.

Table of Contents

Abstract.....	2
1. Introduction.....	4
1.1 Purpose and context.....	4
1.2 Target environment and scope.....	4
1.3 Sources and how claims are classified.....	4
2. What is ROS 2?.....	4
3. Why ROS 2?.....	5
4. ROS 2 Architecture.....	5
4.1 Layers.....	5
4.2 Graph, interfaces and execution.....	6
5. Nodes.....	7
6. Publisher–Subscriber Communication.....	7
6.1 Topic, message type and QoS.....	7
6.2 Timer, callbacks and the frequency correction.....	8
6.3 Command-line inspection and cross-language results.....	9
7. Workspaces and Packages.....	10
7.1 Underlay, overlay and colcon.....	10
7.2 Package types and dependency declarations.....	10
8. Robot Description and URDF.....	11
8.1 Links, joints and origins.....	11
8.2 Xacro, meshes and physical properties.....	13
8.3 Degrees of freedom, gripper and mimic joint.....	14
9. TF and Coordinate Frames.....	14
10. RViz 2.....	15
11. ROS 2 Parameters.....	17
11.1 Concept.....	17
11.2 The example and its correction.....	17
11.3 Boundary between the notes and this preparation.....	18
12. 3-DOF Manipulator Case Study.....	19
12.1 Robot and package responsibilities.....	19
12.2 Implementation choices.....	19
12.3 Demonstration workflow.....	20
12.4 Hardware context.....	20
13. Development Workflow.....	21
14. Advantages of ROS 2.....	22
15. Current Project Status and Future Work.....	22
15.1 Status by category.....	22
15.2 Checks outstanding on the target machine.....	23
15.3 Future work.....	23
16. Conclusion.....	23
References.....	24

1. Introduction

1.1 Purpose and context

The purpose of this report is to explain what ROS 2 is, why it is used and how its main mechanisms work, using a manipulator project that was studied in a practical online course. Rather than describing ROS 2 in the abstract, each concept is connected to a file, a command or a measured result of the prepared project. The course builds a small 3D-printable arm driven by four servo motors and an Arduino board, first as a "digital twin" in software and later as physical hardware [1], [6].

1.2 Target environment and scope

The target platform is Ubuntu 22.04 LTS with ROS 2 Humble Hawksbill, a combination for which Humble is released as Debian packages and which is a Tier 1 platform of that distribution [7], [8]. The scope follows the chain Ubuntu 22.04 → ROS 2 Humble → workspace → packages → nodes → publisher/subscriber communication → robot description → TF → RViz → parameters → manipulator case study. Simulation, motion control, motion planning, voice control and hardware belong to later parts of the course; they are described as context and future work only.

1.3 Sources and how claims are classified

Four categories are used consistently. **Category A – documented student work** is work or study supported by a located piece of evidence in the student's notes [1], [2], [3], [4], [5]. **Category B – instructor material** is code, models and configuration supplied by the course author [6]. **Category C – assignment preparation** covers the changes, verification, figures and documentation produced for this assignment [9]. **Category D – future work** covers activities whose completion is not documented in the supplied notes. Historical screenshots from the notes and new verification results are always identified separately.

2. What is ROS 2?

The official documentation describes the Robot Operating System as "a set of software libraries and tools for building robot applications" [10] and states that, despite its name, it is "not an operating system in the traditional sense" [11]. ROS 2 runs as ordinary user-space software on top of an operating system – here Ubuntu 22.04 – and relies on it for processes, threads, networking and files. What ROS 2 adds is a common way for independent pieces of robot software to find each other, exchange typed data and be configured, together with tools for building, launching and inspecting them.

A running ROS 2 system is described as a *graph*: nodes connected by topics, services, actions and parameters [12]. Nodes "can communicate with other nodes within the same process, in a different process, or on a different machine" [13], and they find each other automatically through the middleware's discovery mechanism [14]. This is what makes ROS 2 suitable for distributed applications: a driver, an algorithm and a visualisation tool can be developed separately and combined at run time, on one computer or several.

ROS 2 is released in *distributions*, versioned sets of packages kept stable after release [15]. Humble Hawksbill, the eighth ROS 2 release, was published on 23 May 2022 and reaches end of life in May 2027 [8], [15]. The notes record these ideas as conceptual study, including that ROS is a framework on top of an operating system (1.2 Course Notes, lessons 16, 17 and 20) [2].

3. Why ROS 2?

The engineering motivation recorded in the notes is reuse: without a common framework, application code is tied to one robot's hardware and must be rewritten for each new platform, whereas a framework with standard interfaces lets the same application logic run on different robots behind hardware abstractions (1.2 Course Notes, lessons 17, 20 and 21) [2]. The project in this report illustrates this on a small scale. The robot-specific part is a model file and a few configuration files; the programs that turn the model into coordinate frames and a 3D view – `robot_state_publisher`, `joint_state_publisher` and `RViz` – are generic Humble packages used unchanged.

ROS 2 was also designed to address limitations of ROS 1 that the notes list for lesson 18: changing network conditions, multi-robot systems, embedded platforms, security and real-time behaviour [2]. Table 1 relates these motivations to what the Humble documentation actually provides and qualifies each point, because several of them are capabilities that must be configured rather than guarantees.

Table 1. Motivations for ROS 2 recorded in the notes and what the Humble documentation provides.

Motivation (notes, lesson 18)	What ROS 2 provides (documentation)	Qualification
Unreliable or changing networks	DDS/RTPS middleware with configurable Quality of Service (history, reliability, durability) [16], [17]	QoS must be chosen and must be compatible between endpoints; it does not guarantee timing.
Many robots and computers	Distributed discovery ("not centralized like in ROS 1") and domain IDs that separate logical networks [16], [18]	All participants must use a compatible middleware and the same domain.
Security	Secured communication between nodes through the middleware's DDS security plug-ins [19]	Security is "turned off by default" and needs security files for each participant.
Real-time behaviour	Designed "with real-time performance constraints in mind" [20]	Requires a real-time kernel and deterministic code; the standard executors are not suited to hard real-time use [21].
Several programming languages	Client libraries <code>rcpp</code> and <code>rcpy</code> share one C core, <code>rcl</code> [22]	Demonstrated in this project for C++ and Python.

4. ROS 2 Architecture

4.1 Layers

ROS 2 is organised in layers (Figure 1). User code is written against a *client library*: `rcpp` for C++ or `rcpy` for Python. Both are built on `rcl`, a common library written in C, so

behaviour that is not language-specific is implemented once [22]. Below `rcl`, the ROS middleware interface (`rmw`) is "the interface between the ROS 2 software stack and the underlying middleware implementation"; the middleware, a DDS or RTPS implementation, is responsible for discovery, publish/subscribe and request/reply mechanics and serialisation [23]. In Humble the default implementation is `rmw_fastrtps_cpp` with eProsima Fast DDS [8], [16]; the test container reported exactly this implementation [9]. The notes contain the same layered view as a text diagram (1.2 Course Notes, lesson 19) [2].

ROS 2 Humble software stack as used by the arduinobot workspace

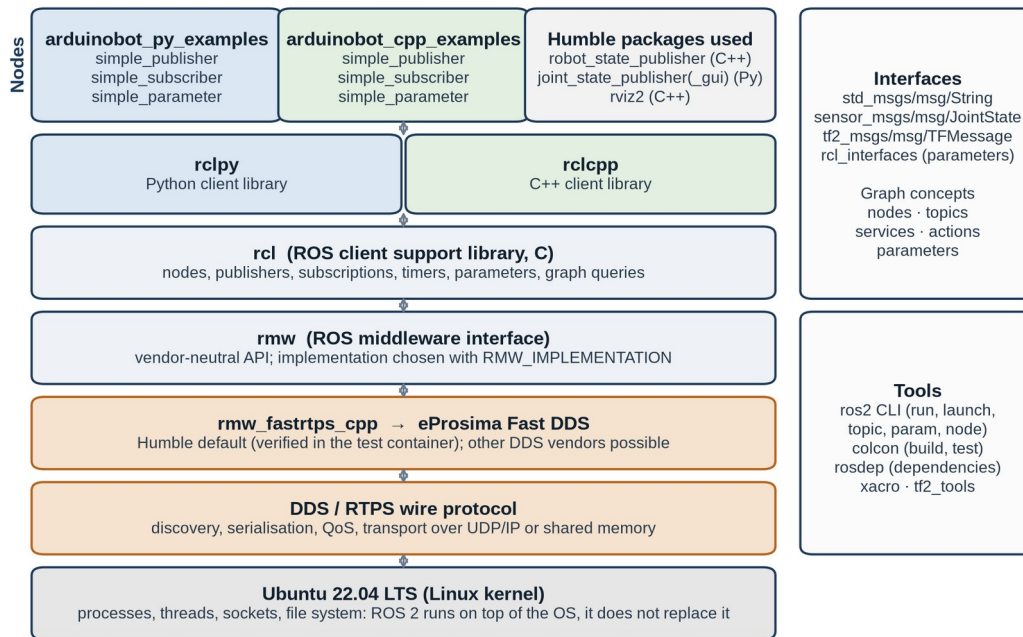


Figure 1. ROS 2 Humble software stack with the nodes of this project. New diagram (category C) based on the documentation cited in Section 4.1; the default RMW was confirmed in the test container.

4.2 Graph, interfaces and execution

Nodes interact through three interface styles: topics for continuous data streams, services for short request/response calls and actions for long-running goals with feedback [24], [25], [26]. Their data types are defined in an interface definition language (.msg, .srv and .action files) from which code is generated for each language [27]; this is why a C++ node and a Python node can exchange a `std_msgs/msg/String` without any conversion code. Parameters are a fourth, node-local mechanism for configuration (Section 11).

Callbacks are executed by an *executor*: calling `spin()` runs a single-threaded executor that waits for timers, messages and other events and invokes the matching callbacks [21]. Discovery is automatic: a node advertises itself on its ROS domain and connections are made only between endpoints with compatible QoS settings [14]. The project stays within this basic model: every example node is spun by one executor, uses the default domain and communicates over topics and the built-in parameter services.

5. Nodes

A node is "a participant in the ROS 2 graph" and typically performs one logical task [13]. A node is not the same thing as its communication endpoints: publishers, subscriptions, timers and parameters are entities *owned* by a node, and one node may own many of them. Nor is a node the same as an operating-system process: an executable "can contain one or more nodes" [12]. The verification illustrates this: while RViz was running, `ros2 node list` showed `/rviz2` and an additional `/transform_listener_impl_...` node created inside the same RViz process [9].

Table 2 lists the nodes that were inspected in the delivered workspace, together with the generic Humble nodes started by the description package.

Table 2. Nodes of the delivered workspace and the Humble nodes it uses.

Executable (package)	Node name	Language	Entities and responsibility
simple_publisher (<code>arduinobot_py_examples</code>)	<code>/simple_publisher</code>	Python	publisher on <code>/chatter</code> , 1 s timer; publishes "Hello ROS 2 - counter: N"
simple_subscriber (<code>arduinobot_py_examples</code>)	<code>/simple_subscriber</code>	Python	subscription on <code>/chatter</code> ; logs "I heard: ..."
simple_parameter (<code>arduinobot_py_examples</code>)	<code>/simple_parameter</code>	Python	parameters <code>simple_int_param</code> (28), <code>simple_string_param</code> ("Antonio"); set-parameter callback
simple_publisher (<code>arduinobot_cpp_examples</code>)	<code>/simple_publisher</code>	C++	publisher on <code>/chatter</code> , 1 s wall timer; publishes "Hello ROS 2 - counter:N"
simple_subscriber (<code>arduinobot_cpp_examples</code>)	<code>/simple_subscriber</code>	C++	subscription on <code>/chatter</code> ; logs "I heard: ..."
simple_parameter (<code>arduinobot_cpp_examples</code>)	<code>/simple_parameter</code>	C++	same parameters and callback contract as the Python node (instructor example)
robot_state_publisher	<code>/robot_state_publisher</code>	C++ (Humble)	reads <code>robot_description</code> , subscribes to <code>/joint_states</code> , publishes <code>/tf</code> , <code>/tf_static</code> , <code>/robot_description</code>
joint_state_publisher(_gui)	<code>/joint_state_publisher</code>	Python (Humble)	publishes <code>/joint_states</code> for all non-fixed joints, from sliders or fixed values
rviz2	<code>/rviz2</code>	C++ (Humble)	3D visualisation of the model and TF

The Python and C++ versions share node names, so each demonstration runs one publisher and one subscriber; to run all nodes together a node can be renamed at start-up, e.g. `--ros-args -r __node:=py_simple_publisher` [12]. The notes document writing, building and running both publishers and both subscribers (1.3, lessons 26–27; 1.4, lessons 28–29) [3], [4].

6. Publisher–Subscriber Communication

6.1 Topic, message type and QoS

Topics implement "a strongly-typed, anonymous publish/subscribe system": publishers and subscribers do not address each other, they only agree on a topic name and a message type [24]. The two must be distinguished. In this project the topic *name* is `/chatter` (created as the relative name `chatter` in the root namespace) and its *type* is `std_msgs/msg/String`, a

message with one string field data. Publishers and subscribers must use the same type to communicate [28]. Figure 2 shows the two cross-language combinations that were exercised.

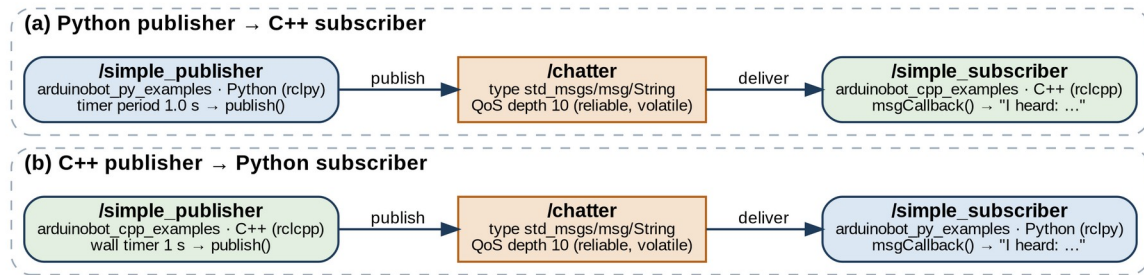


Figure 2. Publisher → topic → subscriber in both cross-language directions. New diagram (category C); names taken from the delivered code. It shows the configuration, not a measured trace.

Both publishers and subscribers pass the integer 10 as their QoS argument. In rclpy and rclcpp this selects a profile with history "keep last" and depth 10; the remaining policies take the defaults, which are "reliable" and "volatile" [17], [29]. `ros2 topic info -v /chatter` in the test container reported RELIABLE and VOLATILE for both endpoints [9]. These are configured policies, not promises: "reliable" makes the middleware retransmit lost samples, but the queue depth still bounds how many samples can wait, and no policy guarantees a delivery time.

6.2 Timer, callbacks and the frequency correction

The publishers are driven by timers. `create_timer()` in rclpy takes a period in seconds; the official tutorial writes `timer_period = 0.5 # seconds` [29]. The instructor's Python publisher stored the value 1.0 in a variable called `frequency_` and passed it directly as the period. At 1.0 the result is the same, but any other "frequency" would have produced the inverse rate. The delivered code keeps `frequency_` as a rate in hertz and passes `1.0 / self.frequency_` (Listing 1); this correction is category C. The same code appears unchanged in the student's notes (1.3 Course Notes, lesson 26, p28–p29) [3].

Listing 1. Python publisher: publisher creation, timer and timer callback
(`arduinobot_py_examples/simple_publisher.py`, delivered version).

```

class SimplePublisher(Node):

    def __init__(self):
        super().__init__("simple_publisher")
        self.pub_ = self.create_publisher(String, "chatter", 10)
        self.counter_ = 0
        self.frequency_ = 1.0 # publishing rate in Hz
        self.get_logger().info("Publishing at %.1f Hz" % self.frequency_)

        # create_timer() expects the timer period in seconds, not a frequency
        self.timer_ = self.create_timer(1.0 / self.frequency_, self.timerCallback)

    def timerCallback(self):
        msg = String()
        msg.data = "Hello ROS 2 - counter: %d" % self.counter_
        self.pub_.publish(msg)
        self.counter_ += 1
  
```

On the receiving side (Listing 2) the C++ subscriber registers `msgCallback` with `create_subscription`. The callback is not called directly by user code; the executor invokes it when `rclcpp::spin()` finds a new message [21].

Listing 2. C++ subscriber: subscription creation and callback

(`arduinobot_cpp_examples/src/simple_subscriber.cpp`, instructor code, unchanged).

```
SimpleSubscriber() : Node("simple_subscriber")
{
    sub_ = create_subscription<std_msgs::msg::String>(
        "chatter", 10, std::bind(&SimpleSubscriber::msgCallback, this, _1));
}

private:
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;

    void msgCallback(const std_msgs::msg::String &msg) const
    {
        RCLCPP_INFO_STREAM(this->get_logger(), "I heard: " << msg.data.c_str());
    }
}
```

6.3 Command-line inspection and cross-language results

The ROS 2 command-line tools inspect the running graph: `ros2 topic list -t` lists topics with their types, `ros2 topic info -v` shows the endpoints and their QoS, `ros2 topic echo` prints messages and `ros2 topic hz` measures the receive rate [28]. The documentation notes that the rate reported by `ros2 topic hz` is measured on its own subscription and "may not exactly match the publisher rate" [28]; the measured values below are therefore observations, while the 1 s timer period is the configured value.

Table 3. Publisher–subscriber runs in the preparation environment (log `verification/logs/run_2026-09-28/20_pubsub.log`).

Scenario (test container)	Publisher	Subscriber	Result
(a) Python → C++	<code>arduinobot_py_examples</code> <code>simple_publisher</code>	<code>arduinobot_cpp_examples</code> <code>simple_subscriber</code>	33 consecutive messages received (counter 0–32, "I heard: Hello ROS 2 - counter: 0" ...); average rate 1.000 Hz; no traceback on Ctrl+C
(b) C++ → Python	<code>arduinobot_cpp_examples</code> <code>simple_publisher</code>	<code>arduinobot_py_examples</code> <code>simple_subscriber</code>	33 consecutive messages received (counter 0–32, "I heard: Hello ROS 2 - counter: 0" ...); average rate 1.000 Hz; no traceback on Ctrl+C

The student's notes contain historical evidence of the same two directions on the student's virtual machine, dated 25 June 2026 by the VM clock: a C++ publisher feeding the Python subscriber (1.4, lesson 28, p22, screenshot IMG020) and the Python publisher feeding the C++ subscriber (1.4, lesson 29, p41–p42, IMG031–IMG032). The notes also show `ros2 topic echo`, `ros2 topic info --verbose` and `ros2 topic hz` measuring 1.000 Hz for both publishers (1.3, lessons 26–27) [3], [4]. These historical runs are category A; the runs in Table 3 are category C. The C++ publisher prints no space after "counter:", while the student's own

C++ version and the Python publisher do; the instructor's string was kept unchanged. This messaging example exchanges text only and does not move the robot model.

7. Workspaces and Packages

7.1 Underlay, overlay and colcon

A workspace is "a directory containing ROS 2 packages" [30]. The ROS 2 installation in `/opt/ros/humble` acts as the *underlay*; a local workspace sourced on top of it is an *overlay*, whose packages take precedence over those of the underlay [31], [30]. Sourcing `install/setup.bash` adds the overlay and the underlay it was built against to the current shell only, so it must be repeated in each new terminal [30]. The notes document this sequence on the student's machine: `~/arduinobot_ws/src`, a first `colcon build`, creation of both example packages, a rebuild, sourcing and `ros2 pkg list` showing the new packages (1.3 Course Notes, lesson 25) [3].

`colcon` builds the packages in dependency order [32]; with `--symlink-install`, Python modules and launch files are installed as links to the source tree and can be edited without reinstalling [33]. The instructor's repository holds seven snapshots of the same workspace, one per course section, with identical package names; only the Section 4 snapshot was placed in `src` [6], [9].

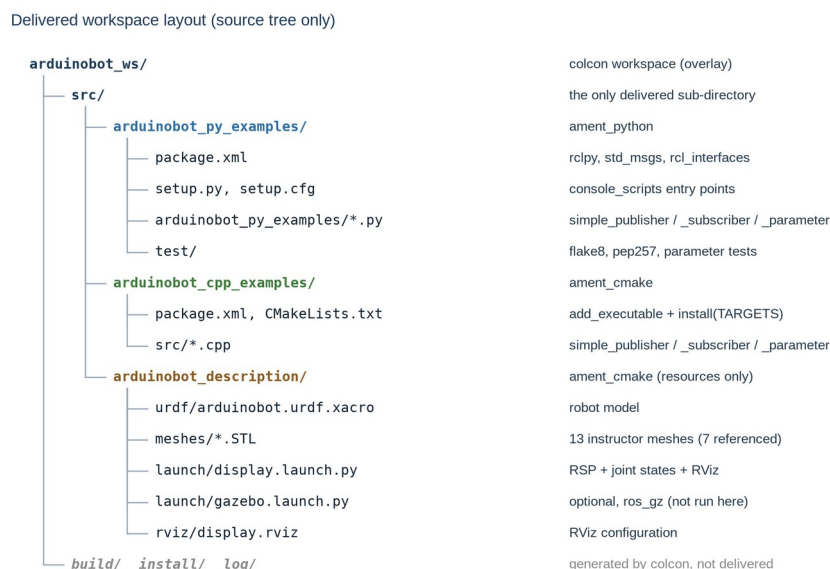


Figure 3. Delivered workspace (source tree). `build/`, `install/` and `log/` are generated by `colcon`. New figure (category C).

7.2 Package types and dependency declarations

Every package has a `package.xml` manifest that declares its dependencies [32]. Two build types are used (Table 4). In the `ament_python` package, `setup.py` registers console-script entry points such as `simple_publisher = arduinobot_py_examples.simple_publisher:main`, and `setup.cfg` installs them where `ros2 run` looks for executables [29], [34]. In the `ament_cmake` packages,

`add_executable`, `ament_target_dependencies` and `install(TARGETS ...)` build and install the C++ programs [35], and `install(DIRECTORY launch meshes urdf rviz ...)` copies the description's resources into the package's share directory, where `package://` URIs and `get_package_share_directory()` find them.

Table 4. Packages of the delivered workspace.

Package	Build type	Declared dependencies (delivered)	Installs
arduinobot_py_examples	ament_python	exec: rclpy, std_msgs, rcl_interfaces; test: ament_copyright, ament_flake8, ament_pep257, python3-pytest	3 console scripts
arduinobot_cpp_examples	ament_cmake	depend: rclcpp, std_msgs, rcl_interfaces	3 executables in <code>lib/arduinobot_cpp_examples</code>
arduinobot_description	ament_cmake	exec: ament_index_python, launch, launch_ros, robot_state_publisher, urdf, joint_state_publisher, joint_state_publisher_gui, rviz2, xacro, ros2launch; ros_gz_sim, ros_gz_bridge (optional Gazebo launch only)	<code>launch/</code> , <code>meshes/</code> , <code>urdf/</code> , <code>rviz/</code>

Direct dependencies should be declared even when they arrive transitively; the launch files' imports `ament_index_python`, `launch` and `launch_ros` were therefore added to the manifest (category C). `rosdep` resolves declared keys to system packages [36]; in the test container it reported only `ros_gz_sim` and `ros_gz_bridge` missing, which only the optional Gazebo launch file needs [9].

8. Robot Description and URDF

8.1 Links, joints and origins

The Unified Robot Description Format (URDF) is an XML format "for specifying the geometry and organization of robots" [37]. A model is a tree of *links* (rigid bodies) connected by *joints*; each joint names one parent and one child link, so the tree has exactly one root [38]. The arduinobot model has eight links and seven joints (Figure 4, Table 5), all read from the expanded file of the delivered package [9].

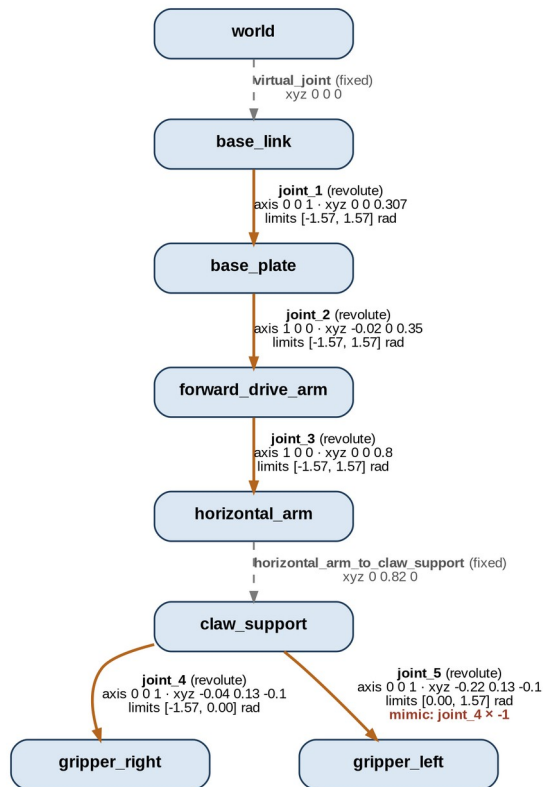


Figure 4. Link-joint tree of `arduinobot.urdf.xacro`. Dashed edges: fixed joints; `joint_5` mimics `joint_4` ($\times -1$). Generated from the verified model (category C); replaces the instructor's 2021 `arduinobot.pdf`, which still shows a removed `tool_link`.

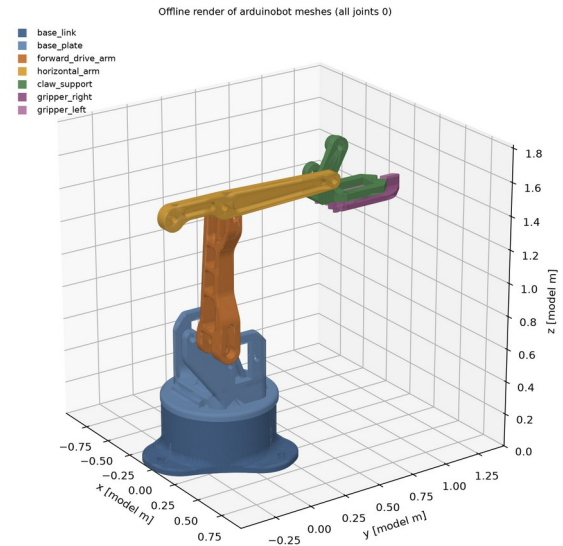


Figure 5. Offline render of the instructor's meshes placed with the URDF kinematics (all joints 0). Rendered with matplotlib from the expanded URDF (category C); not an RViz test. Axes in model metres.

Table 5. Joints of the model (all revolute joints: effort 30, velocity 10 as supplied Xacro properties).

Joint	Type	Parent → child	Origin xyz (m)	Axis	Limits (rad)
virtual_joint	fixed	world → base_link	0 0 0	—	—
joint_1	revolute	base_link → base_plate	0 0 0.307	z	$-\pi/2 \dots \pi/2$
joint_2	revolute	base_plate → forward_drive_arm	-0.02 0 0.35	x	$-\pi/2 \dots \pi/2$
joint_3	revolute	forward_drive_arm → horizontal_arm	0 0 0.8	x	$-\pi/2 \dots \pi/2$
horizontal_arm_to_claw_support	fixed	horizontal_arm → claw_support	0 0.82 0	—	—
joint_4	revolute	claw_support → gripper_right	-0.04 0.13 -0.1	z	$-\pi/2 \dots 0$
joint_5	revolute (mimic)	claw_support → gripper_left	-0.22 0.13 -0.1	z	$0 \dots \pi/2; = -1 \times \text{joint}_4$

Two kinds of origin must be distinguished. A joint `<origin>` places the child link's frame relative to the parent link's frame [38]; it defines the kinematics. The `<origin>` inside a link's `<visual>` or `<collision>` element only places the geometry relative to the link's own frame [38]. Listing 3 shows both for `forward_drive_arm`: `joint_2` puts the shoulder frame 0.35 m above the base plate and rotates it about the x axis, while the visual origin rotates and shifts the STL mesh so that it lines up with that frame. The notes record exactly this distinction

when the student corrected the base mesh offset with a visual origin of $(-0.5, -0.5, 0)$ (1.4, lesson 32, p62–p64) [4].

Listing 3. A link with its visual element and the joint that moves it (arduinobot_description/urdf/arduinobot.urdf.xacro, instructor file, unchanged; the repeated collision element is abbreviated).

```
<link name="forward_drive_arm">
  <xacro:default_inertial mass="0.1"/>
  <visual>
    <origin rpy=" 0 -${PI / 2} ${PI / 2}" xyz="0.19 0.06 -0.08"/>
    <geometry>
      <mesh
filename="package://arduinobot_description/meshes/forward_drive_arm.STL"
scale="0.01 0.01 0.01"/>
    </geometry>
  </visual>
  <!-- collision element identical to the visual -->
</link>

<joint name="joint_2" type="revolute">
  <parent link="base_plate"/>
  <child link="forward_drive_arm"/>
  <origin xyz="-0.02 0 0.35"/>
  <axis xyz="1 0 0"/>
  <limit lower="-${PI / 2}" upper="${PI / 2}"
        effort="${effort}" velocity="${velocity}"/>
</joint>
```

8.2 Xacro, meshes and physical properties

The file is a Xacro document: properties such as π , effort and velocity, expressions like $\pi / 2$ and a macro `default_inertial` are expanded by the xacro program into plain URDF [39]. The launch file runs xacro through a Command substitution and passes the result to `robot_state_publisher` as the `robot_description` parameter, the pattern shown in the documentation [39]. Seven STL meshes are referenced with `package://arduinobot_description/meshes/<name>.STL` URIs; the verification resolved all fourteen visual and collision references with exact upper-case extensions [9].

`<collision>` elements define geometry for collision checking and `<inertial>` elements define mass, centre of mass and inertia for physics simulation [40]. In this model the collision geometry reuses the detailed visual meshes, and every link has the placeholder inertia $ixx = iyy = izz = 1.0$ whatever its mass; effort 30 and velocity 10 are uniform Xacro properties. These are supplied model parameters, not measured properties of the robot, and the model is therefore a kinematic and visual description rather than a validated dynamic model. The documentation itself notes that velocity and effort values in its example "don't matter" for visualisation [41].

The mesh files carry no unit. Their coordinates are consistent with millimetres (for example, pivot bores 80.0 units apart), but the model scales them by 0.01, so one unit becomes one centimetre and the rendered arm is about 1.69 m tall (Figure 5). All joint origins were written for that scale: `joint_3` at 0.8 m equals the 80-unit bore spacing. Course diagrams reproduced in the notes give the corresponding real offsets in centimetres, e.g. 3.07 cm where the URDF uses 0.307 m (1.4, lesson 32, p84–p89) [4], [6]. The software model is therefore about ten

times the physical size. It was deliberately not rescaled, because a correct rescale would require changing every origin and no measured dimensions of a built robot are available; model values in this report are quoted as model values.

8.3 Degrees of freedom, gripper and mimic joint

The URDF contains five revolute joints, but the robot is not a five-degree-of-freedom arm. The arm has three independent degrees of freedom – base rotation (`joint_1`), shoulder (`joint_2`) and elbow (`joint_3`) – which position the gripper, as the notes explain for the course robot (1.1 Course Notes, lesson 5, p6) [1]. `joint_4` opens and closes the right finger; it is a gripper actuation, not an additional arm DOF. `joint_5` moves the left finger but carries `<mimic joint="joint_4" multiplier="-1"/>`, so its position is always derived from `joint_4` and it adds no command variable. Consequently only four joints are commanded independently: `joint_state_publisher_gui` shows four sliders, and in the verification `joint_4 = -0.6` produced `joint_5 = 0.6` in `/joint_states` [9]. The physical robot of the course uses a parallelogram linkage to keep the gripper orientation (notes, lesson 5) [1]; the URDF does not model that linkage and treats the arm as a serial chain, and six supplied meshes of linkage parts are unused.

9. TF and Coordinate Frames

tf2 "lets the user keep track of multiple coordinate frames over time" and maintains them as a tree [42]. Each URDF link becomes a frame. `robot_state_publisher` reads the model and publishes the transforms: those of fixed joints once on `/tf_static` with transient-local durability [8], and those of movable joints on `/tf` whenever new joint positions arrive on `/joint_states` [43]. Joint positions come from `joint_state_publisher` or its GUI, which parse the model, publish values for all non-fixed joints and let `robot_state_publisher` "calculate all of the transforms" [41]. Figure 6 shows these relationships for the delivered launch file.

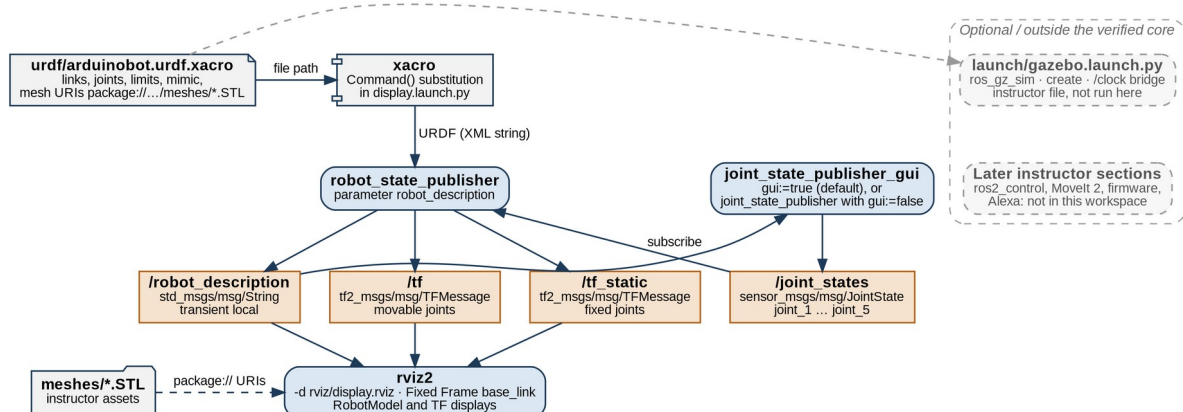


Figure 6. Software architecture of the manipulator visualisation (`display.launch.py`). Dashed grey elements are optional or outside the verified core. New diagram (category C) derived from the delivered launch file and the topics observed in the verification.

In the verification, `joint_state_publisher` was started with fixed values (`joint_1 = 0.5`, `joint_2 = 0.3`, `joint_3 = -0.4`, `joint_4 = -0.6`) and published at a measured 9.99 Hz, while `ros2 run tf2_tools view_frames` recorded the tree in Figure 7: the two fixed joints appear as static transforms and the five movable joints at about 10 Hz [44], [9]. `ros2 run tf2_ros tf2_echo world claw_support` returned the translation $(-0.295, 0.499, 1.339)$ m and roll-pitch-yaw $(-0.100, 0.000, 0.500)$ rad [9]. An independent forward-kinematics calculation from the joint origins and axes of Table 5 gives the same values, which confirms that the published TF tree matches the model: yaw equals `joint_1` and roll equals `joint_2 + joint_3`, because both pitch joints rotate about x.

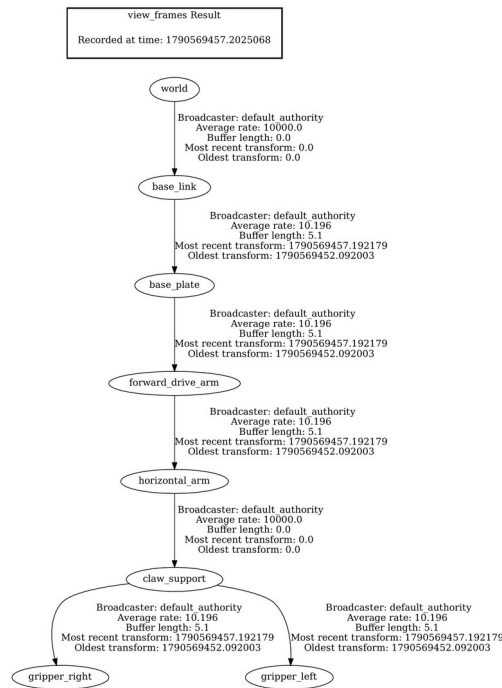


Figure 7. TF tree recorded by `view_frames` during the verification. Genuine output of `ros2 run tf2_tools view_frames` (category C, verification/logs/run_2026-09-28), converted from PDF.

10. RViz 2

RViz is "a 3D visualizer for the Robot Operating System" [45]. It shows *displays*: the RobotModel display draws the robot "in the correct pose (as defined by the current TF transforms)" and the TF display draws the frame hierarchy [45]. All data are transformed into the *fixed frame*, which "should not be moving relative to the world" [45]. The delivered configuration `display.rviz` uses the fixed frame `base_link`; because `virtual_joint` fixes `base_link` to `world` with a zero offset, this is equivalent to the tree's root frame.

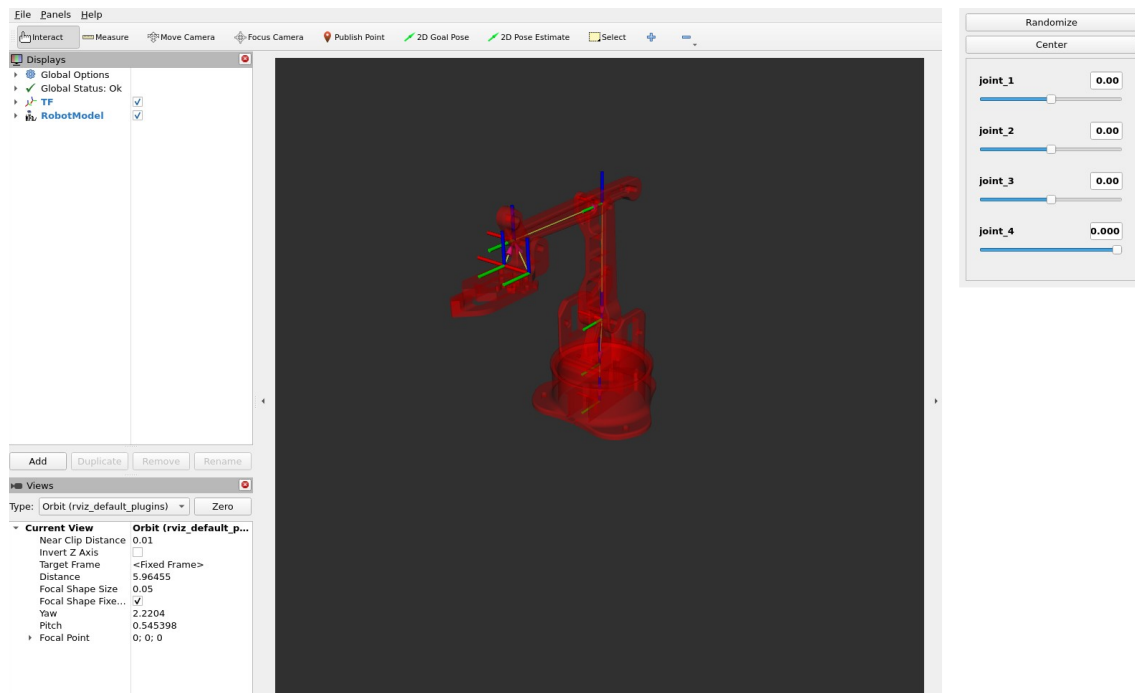


Figure 8. `ros2 launch arduinobot_description display.launch.py` in the preparation environment: RViz with RobotModel and TF displays, and the joint_state_publisher_gui window with four sliders. Genuine screenshot of the delivered workspace (category C), Ubuntu 22.04 / Humble container, virtual X server with software OpenGL; 28 Sep. 2026.

The RobotModel display obtains the model from the `/robot_description` topic, which `robot_state_publisher` publishes with transient-local durability, so that a subscriber started later still receives it – provided both sides use transient local [17]. The instructor's configuration requested "Volatile". With the unmodified configuration and RViz 11.2.28, RViz started eight seconds after `robot_state_publisher` still displayed the model, and `ros2 topic info -v` showed that it had in fact subscribed with TRANSIENT_LOCAL (baseline log); the value was changed to "Transient Local" so that the file states the QoS actually used, and the late start was repeated with the delivered file (Figure 9). Three stale entries for a link `tool_link`, which the model no longer contains, were removed from the same file [9]. The URDF defines no `<material>` elements, and RViz drew the meshes in red, as in the student's own screenshot.

RViz is a visualisation tool and not a simulator. It draws the pose implied by TF; it computes no forces, gravity, contacts or motor dynamics, and a slider moves the drawn model instantly whether or not a real arm could follow. Physical simulation is the task of Gazebo, which the course introduces with the optional `gazebo.launch.py`; its Time panel is mainly useful "when running in a simulator" [45]. The notes contain a historical RViz screenshot of the complete model, produced with `ros2 launch urdf_tutorial display.launch.py model:=.../arduinobot.urdf.xacro` on 3 July 2026 by the VM clock (Figure 10) [4]. It shows that the student's own model loaded with all meshes; it used the `urdf_tutorial` launch file rather than the instructor's `display.launch.py`, and joint movement is not documented. The conceptual RViz lesson in 1.5 contains no practical run [5].

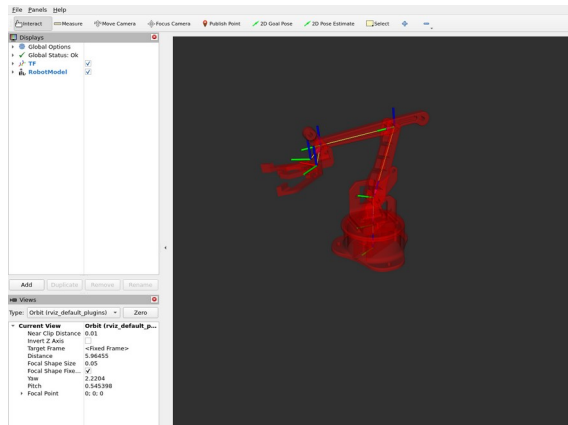


Figure 9. Preparation run: RViz started 8 s after robot_state_publisher; joint_1 = -0.5, joint_2 = 0.3, joint_3 = -0.4, joint_4 = -0.6 (gripper open). Genuine screenshot of the delivered workspace (category C), 28 Sep. 2026.

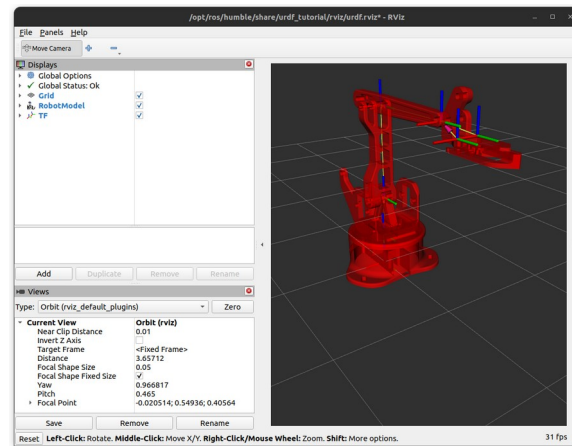


Figure 10. Historical screenshot from the student's notes: the student's model in RViz (urdf_tutorial configuration). Extracted from 1.4.docx, lesson 32, p79, IMG055 (category A, historical evidence; not a test of the delivered workspace).

11. ROS 2 Parameters

11.1 Concept

Parameters "are associated with individual nodes" and configure them "at startup (and during runtime), without changing the code"; each consists of a key, a typed value and a descriptor [46]. There is no global parameter server: each node maintains its own parameters [47] and exposes parameter services through which `ros2 param` lists, reads and sets them [46]. A node must normally declare the parameters it accepts, and by default "attempts to change the type of a declared parameter at runtime will fail" [46]. A node can also register a "set parameter" callback whose purpose is "to inspect the upcoming change ... and explicitly reject the change"; such callbacks should have no side effects [46]. The notes record the concept, including the per-node nature of parameters (1.5, lesson 35) [5].

11.2 The example and its correction

The `simple_parameter` node declares `simple_int_param` (default 28) and `simple_string_param` (default "Antonio") and registers `paramChangeCallback` (Listing 4). The instructor's callback started from a default `SetParametersResult`, whose successful field is false, and set it to true only when one of its own parameters appeared. Two consequences were reproduced on the unmodified code: `ros2 param set /simple_parameter use_sim_time true` was rejected, although `use_sim_time` is a standard parameter of every node [47]; and the same change was accepted when sent together with a valid `simple_int_param` in one atomic request [9]. The delivered callback first validates the whole request, rejects it with a reason if one of the node's parameters has the wrong type, accepts parameters it does not manage and only then logs the changes. In Humble, `rcplpy` enforces the declared type before calling the callback, so the callback's own

type check is a second line of defence; this is stated in the code and exercised by a unit test. The same correction was applied to the C++ node.

Listing 4. Parameter declaration and set-parameter callback

(`arduinobot_py_examples/simple_parameter.py`, delivered version; category C correction of instructor code).

```
def __init__(self):
    super().__init__("simple_parameter")
    self.declare_parameter("simple_int_param", 28)
    self.declare_parameter("simple_string_param", "Antonio")

    self.add_on_set_parameters_callback(self.paramChangeCallback)

def paramChangeCallback(self, params):
    # Check the whole request first, so that one valid parameter cannot
    # make an update containing an invalid one succeed.
    for param in params:
        expected_type = self.EXPECTED_TYPES.get(param.name)
        if expected_type is not None and param.type_ != expected_type:
            return SetParametersResult(
                successful=False,
                reason="%s must be of type %s" % (param.name, expected_type.name))

    # Parameters not handled here (e.g. use_sim_time) are accepted unchanged
    for param in params:
        if param.name == "simple_int_param":
            self.get_logger().info(
                "Param simple_int_param changed! New value is %d" % param.value)
        elif param.name == "simple_string_param":
            self.get_logger().info(
                "Param simple_string_param changed! New value is %s" %
param.value)

    return SetParametersResult(successful=True)
```

Table 6. Parameter behaviour of the delivered Python node (verification/logs/run_2026-09-28/30_parameters.log, 10_build.log).

Request (Python node, test container)	Result
ros2 param get ... simple_int_param/ simple_string_param	28 / "Antonio" (defaults)
ros2 param set ... simple_int_param 30, ... simple_string_param "ROS 2"	successful; node logs both changes
ros2 param set ... simple_int_param hello	rejected: "Wrong parameter type, expected 'Type.INTEGER' got 'Type.STRING'"
ros2 param set ... use_sim_time true	successful (rejected by the original callback)
atomic request: simple_int_param = 5 with an integer for simple_string_param	rejected as a whole; values stay 30 and "ROS 2"
start-up override --ros-args -p simple_int_param:=42	value 42
colcon test (6 parameter unit tests, flake8, pep257)	passed

11.3 Boundary between the notes and this preparation

The Python parameter laboratory is only partly documented. The notes show the creation of an empty `simple_parameter.py` in the student's workspace and a full listing of the node with the instructor's names and defaults (1.5, lesson 36, p7–p10) [5]. The listing is unindented as stored in the document and logs the string parameter with `%d`, which would raise a `TypeError` when that parameter is set; the delivered code uses `%s`, as the instructor's file

does. No entry-point registration, rebuild, run or `ros2 param` output is documented, and no C++ parameter lesson appears in the notes. The runnable parameter demonstration and its results therefore belong to category C, and completing the laboratory on the student's machine is future work.

12. 3-DOF Manipulator Case Study

12.1 Robot and package responsibilities

The course robot is a 3D-printable desktop arm with three joints – base, shoulder and elbow – plus a gripper, driven by four hobby servo motors ("typically $4 \times$ SG90") through an Arduino board (1.1 Course Notes, lesson 5, p5–p7) [1]. In software the robot is represented by the `arduinobot_description` package; the two example packages teach the communication and configuration mechanisms that later course sections use to control it. Responsibilities are deliberately separated: the description package contains no program code, only the model, its meshes, a launch file and an RViz configuration, while all running programs in the visualisation are generic Humble nodes.

12.2 Implementation choices

- **Baseline.** Section 4 on main (commit 4936385) is the smallest snapshot with all three packages, and its model does not depend on the control package, unlike Sections 5–9 [6], [9]. Branch `gz-classic` targets Gazebo Classic, whereas the notes record the installation of the modern-Gazebo packages `ros-humble-ros-gz*` (1.2, lesson 14) [2].
- **Minimal corrections.** Twelve changes (C1–C12) were made; the robot model, meshes, topic, message type, node names and defaults are unchanged. Table 7 summarises them; the full diff is delivered in patches/.
- **Joint-state source.** A gui launch argument selects exactly one of `joint_state_publisher_gui` (default) and `joint_state_publisher`, so the pipeline can run without a slider window and two conflicting `/joint_states` publishers cannot be started together.
- **No combined launch file.** Messaging and visualisation are independent; combining them would couple the example and description packages without demonstrating anything new.
- **Simulation kept optional.** `gazebo.launch.py` was kept as instructor material with formatting fixes only. It was not executed, and on Humble its use of `GZ_SIM_RESOURCE_PATH` must be checked against the installed Gazebo Fortress version [9].

Table 7. Changes made to the instructor snapshot (details in `docs/PROVENANCE.md`).

ID	Change (category C)	Reason
C1	timer period <code>1.0 / frequency_</code> ; <code>log "%.1f Hz"</code>	<code>create_timer</code> expects a period in seconds
C2	clean Ctrl+C handling in the three Python nodes	tracebacks and "exited with failure" on shutdown
C3	subscriber variable name, no-op statement removed	readability
C4–C5	parameter callback validates the whole request; unmanaged parameters accepted (Python, C++)	<code>use_sim_time</code> rejected; inconsistent batch results

ID	Change (category C)	Reason
C6	six pytest cases for the parameter contract	guard the corrected behaviour
C7	flake8 style fixes in the Python package	package's own linter test failed
C8	gui launch argument (default unchanged)	headless runs; single joint-state source
C9	display.rviz: stale tool_link removed, description QoS "Transient Local"	link absent from model; configuration states the QoS used
C10	direct exec dependencies declared (launch, launch_ros, ament_index_python, joint_state_publisher)	imports and C8
C11–C12	copyright linter skipped (no per-file headers); lint-only fixes in gazebo.launch.py	linter failures; launch logic unchanged

12.3 Demonstration workflow

After building and sourcing the workspace, the demonstration consists of four independent steps, each documented with exact commands in the README: (1) Python publisher with C++ subscriber, (2) C++ publisher with Python subscriber, (3) the parameter node inspected and updated with `ros2 param`, and (4) `ros2 launch arduinobot_description display.launch.py`, moving the sliders and inspecting `/joint_states` and TF. Listing 5 shows the part of the launch file that loads the model and selects the joint-state source.

Listing 5. Model loading and joint-state source selection

(`arduinobot_description/launch/display.launch.py`; the gui condition is a category C addition).

```
robot_description = ParameterValue(Command(["xacro ",
LaunchConfiguration("model")]),
                                value_type=str)

robot_state_publisher_node = Node(
    package="robot_state_publisher",
    executable="robot_state_publisher",
    parameters=[{"robot_description": robot_description}]
)

# Exactly one joint state source is started, selected by the "gui" argument
joint_state_publisher_gui_node = Node(
    package="joint_state_publisher_gui",
    executable="joint_state_publisher_gui",
    condition=IfCondition(LaunchConfiguration("gui"))
)

joint_state_publisher_node = Node(
    package="joint_state_publisher",
    executable="joint_state_publisher",
    condition=UnlessCondition(LaunchConfiguration("gui"))
)
```

12.4 Hardware context

The physical robot was not built or used in this preparation, and the notes contain no evidence of assembly, wiring or firmware upload. The repository's later sections state the following facts, which are reported only as file contents: the Arduino sketch attaches the base, shoulder, elbow and gripper servos to pins 8, 9, 10 and 11 and opens the serial port at 115 200 baud (Section9_Build/.../robot_control.ino); the `ros2_control` hardware interface sends frames of the form `b%03d,s%03d,e%03d,g%03d`, to `/dev/ttyACM0` [6]. No file specifies the board model, wiring, supply voltage or power arrangement, so none is stated here.

13. Development Workflow

The work followed the cycle in Figure 11: inspect the sources, prepare the packages, resolve dependencies, build, source, run, inspect and document, returning to preparation when a check revealed a defect. Every check was scripted (verification/scripts/), so that it can be repeated on the target machine and its output kept as evidence.

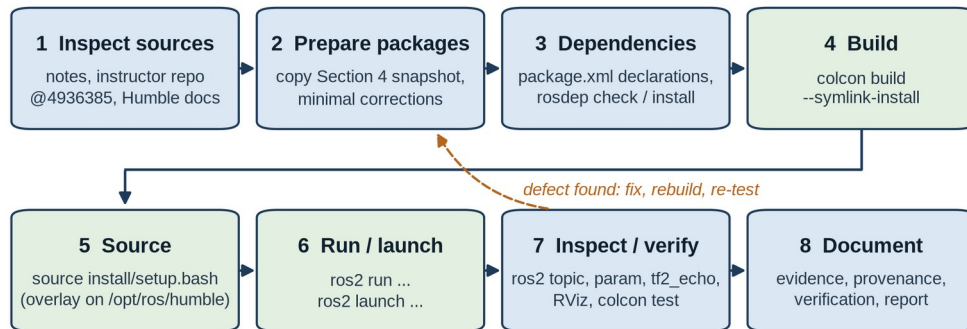


Figure 11. Development and verification workflow. New diagram (category C).

The preparation environment was a cloud container running Ubuntu 24.04, not the target system. The ROS 2 checks were therefore run inside the official `osrf/ros:humble-desktop` Docker image (Ubuntu 22.04.5, `roscpp` 16.0.19, `roslpy` 3.3.21, `rviz2` 11.2.28). Because the network policy blocked `packages.ros.org`, `xacro` 2.1.1 and `joint_state_publisher` 2.4.0 – the versions the notes show being installed with `apt` – were built from their upstream release tags in a separate underlay. RViz ran on a virtual X server with software OpenGL [9].

Table 8. Verification summary (details and logs in `docs/VERIFICATION.md`).

Check	Method	Status
Layout, manifests, launch/Xacro syntax, portable paths	static inspection, xacro, check_urdf	passed (static)
package.xml schema (format 3)	xmllint with the REP 149 schema	passed (static)
Build of all three packages	colcon build --symlink-install	passed (build)
Package tests (linters, 6 parameter tests)	colcon test; xmllint could not download its schema	passed except xmllint (blocked by environment)
Python → C++ and C++ → Python on /chatter	ros2 run, ros2 topic list/info/echo/hz	passed (runtime)
Parameters (Python and C++)	ros2 param, atomic service call	passed (runtime)
Model tree, joints, mimic, 14 mesh URIs	check_model.py, check_urdf	passed (static)
Joint states, TF, static TF	ros2 topic echo/hz, tf2_echo, view_frames	passed (runtime)
RViz display, late join, clean shutdown	display.launch.py under Xvfb, screenshots	passed (runtime, software rendering)
Gazebo launch, ros2_control, MoveIt, hardware	–	not tested

Typical problems met during the work: a terminal without `source install/setup.bash` does not find the packages; CLI tools started with `--no-daemon` sometimes listed no nodes

before discovery finished (`--spin-time 5` fixed this); `ros2 topic echo /tf_static` waited until its QoS was set to reliable and transient local like the publisher; and on Ctrl+C launch reports `joint_state_publisher_gui` as "died" (exit code -2), an upstream behaviour, while `robot_state_publisher` and RViz finish cleanly [9].

14. Advantages of ROS 2

The project demonstrates several advantages directly:

- **Language interoperability.** Python and C++ nodes exchanged messages in both directions without conversion code, because both client libraries use generated interface code and a common core [22].
- **Reuse of generic components.** `robot_state_publisher`, `joint_state_publisher` and RViz visualised a robot they know nothing about, driven only by its URDF [41].
- **Introspection.** The running graph, topic types, QoS, parameters and transforms could be inspected with standard tools, which made every verification step observable.
- **Configuration without recompilation.** Parameters changed node behaviour at start-up and at run time, and launch arguments selected the joint-state source.
- **Reproducible builds.** Declared dependencies, `colcon` and `rosdep` allowed the same workspace to be built from source in a clean container [36].

The same project also shows practical limitations. The environment must be sourced correctly in every terminal; discovery and QoS compatibility introduce timing effects that beginners do not expect; and the ecosystem moves quickly – the course repository already contains two Gazebo generations, renamed plug-ins and environment variables, and Humble reaches end of life in May 2027 [15]. ROS 2 does not by itself make an application real-time, secure or compatible with arbitrary hardware: real-time behaviour needs a suitable kernel and deterministic code [20], security must be enabled and configured [19], and hardware support requires a driver or `ros2_control` interface such as the one the course writes for its Arduino.

15. Current Project Status and Future Work

15.1 Status by category

Table 9. Project status by category.

Category	Status
A – Documented student work	ROS 2 Humble and tools installed; development environment configured; workspace and both example packages created; Python and C++ publishers and subscribers built and run; topic inspection; both cross-language directions; description package and complete URDF/Xacro model visualised in RViz with <code>urdf_tutorial</code> ; conceptual study of ROS 2 architecture, communication, packages, RViz and parameters; Python parameter lab started (code listing only). Sources: notes 1.1–1.5 [1], [2], [3], [4], [5].
B – Instructor material	Section 4 packages (examples, parameter nodes, model, meshes, launch and RViz files), later sections with <code>ros2_control</code> , MoveIt 2, application, Alexa and firmware [6].
C – Assignment preparation	Clean workspace with corrections C1–C12; build, tests and runtime checks in an Ubuntu 22.04 / Humble container; parameter demonstration; TF and RViz verification; figures, documentation and this report [9].

Category	Status
D – Future work (not documented in the notes)	Completing the Python parameter lab; C++ parameters; practical services and actions; the instructor's display launch on the student's machine; Gazebo simulation; ros2_control; MoveIt 2; the application and Alexa integration; building and wiring the physical robot.

15.2 Checks outstanding on the target machine

The preparation verified the project in a container that uses the target distribution but not the student's machine. The following remain to be run on Ubuntu 22.04 with Humble installed from apt, using the commands in the README and `verification/scripts/`:

- `rosdep install` and `colcon build --symlink-install` in `~/arduinobot_ws`, and `colcon test` with network access (so that the `xmllint` test can download its schema);
- the two cross-language runs and the parameter commands in separate terminals;
- `display.launch.py` with a real display and hardware-accelerated graphics, including moving the sliders and observing the mimic finger;
- optionally `gazebo.launch.py`, checking whether the meshes resolve with the installed Gazebo Fortress version.

15.3 Future work

The natural continuation follows the course: completing and running the parameter laboratory, then services and actions, simulation in Gazebo, `ros2_control` controllers, MoveIt 2 motion planning, the task server and voice interface, and finally the physical robot. Two technical extensions follow from this report: a real-scale model (mesh scale 0.001 with all origins divided by ten) together with inertial values computed from the meshes, and a migration plan to a supported distribution before Humble's end of life in May 2027 [15].

16. Conclusion

ROS 2 is a framework of libraries and tools that runs on top of an operating system and organises robot software as a graph of nodes communicating through typed topics, services and actions over a DDS-based middleware, configured through parameters and launch files. The arduinobot project made each of these concepts concrete: two small programs in different languages communicating over `/chatter`, a parameter node whose update rules had to be corrected, a workspace with three packages, and a URDF model that generic Humble nodes turn into coordinate frames and a 3D view.

The student's notes document installation, workspace and package creation, publishers and subscribers in both languages including both cross-language directions, and a complete robot model visualised in RViz; the parameter laboratory and all later course stages are not documented as completed. The prepared workspace preserves the instructor's robot model, adds twelve documented corrections, and was built and exercised in an Ubuntu 22.04 / ROS 2 Humble container. Its behaviour on the student's own machine, simulation, control, motion planning and hardware remain to be verified, and the model is a ten-times-scaled kinematic description with placeholder dynamics. Within these limits, the project shows that ROS 2's

value lies less in any single feature than in the combination of standard interfaces, reusable components and inspection tools that allowed every step to be checked.

References

Pages of the ROS 2 Humble documentation were read from the documentation source repository (ros2/ros2_documentation, branch humble, commit 35b00f1f3c1ab7c14bf85e35fa895f9f580ea279, inspected 28 Sep. 2026) because docs.ros.org was not reachable from the preparation environment; the URLs are those at which the same pages are published. References [1]–[5] are evidence of the student's course progress; the others support the technical content.

- [1] [Student Name], "1.1 Course Notes," personal course notes for *Robotics and ROS 2 – Learn by Doing! Manipulators*, lessons 1, 4–6, Microsoft Word 97-2003 document, last saved 4 Jun. 2026 (file metadata), unpublished.
- [2] [Student Name], "1.2 Course Notes," personal course notes, lessons 11, 14, 16–24, Microsoft Word 97-2003 document, last saved 6 Jun. 2026 (file metadata), unpublished.
- [3] [Student Name], "1.3 Course Notes," personal course notes, lessons 25–27, Microsoft Word 97-2003 document, last saved 25 Jun. 2026 (file metadata), unpublished.
- [4] [Student Name], "1.4," personal course notes, lessons 28–32, Microsoft Word (OOXML) document, last modified 9 Jul. 2026 (file metadata), unpublished.
- [5] [Student Name], "1.5," personal course notes, lessons 34–36, Microsoft Word 97-2003 document, last saved 3 Aug. 2026 (file metadata), unpublished.
- [6] A. Brandi, "Robotics-and-ROS-2-Learn-by-Doing-Manipulators," GitHub repository, branch main, commit 4936385347e477c59927382c904883c432c9b33d, retrieved 28 Sep. 2026. [Online]. Available: <https://github.com/AntoBrandi/Robotics-and-ROS-2-Learn-by-Doing-Manipulators>
- [7] "Ubuntu (deb packages)," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- [8] "Humble Hawksbill (humble)," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Releases/Release-Humble-Hawksbill.html>
- [9] Assignment preparation material: workspace arduinobot_ws, docs/ (provenance, course-progress evidence, repository audit, verification) and verification/logs/run_2026-09-28, delivered with this report.
- [10] "ROS 2 Documentation," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/index.html>
- [11] "About ROS," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/About-ROS.html>
- [12] "Understanding nodes," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>
- [13] "Nodes," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html>
- [14] "Discovery," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Discovery.html>
- [15] "Distributions," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Releases.html>
- [16] "Different ROS 2 middleware vendors," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Different-Middleware-Vendors.html>
- [17] "Quality of Service settings," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html>
- [18] "The ROS_DOMAIN_ID," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Domain-ID.html>
- [19] "ROS 2 Security," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Security.html>
- [20] "Understanding real-time programming," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Demos/Real-Time-Programming.html>
- [21] "Executors," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Executors.html>
- [22] "Client libraries," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Client-Libraries.html>
- [23] "Internal ROS 2 interfaces," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Advanced/About-Internal-Interfaces.html>
- [24] "Topics," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html>
- [25] "Services," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Services.html>

- [26] "Actions," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Actions.html>
- [27] "Interfaces," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html>
- [28] "Understanding topics," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>
- [29] "Writing a simple publisher and subscriber (Python)," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>
- [30] "Creating a workspace," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>
- [31] "Configuring environment," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html>
- [32] "The build system," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Advanced/About-Build-System.html>
- [33] "Using colcon to build packages," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html>
- [34] "Creating a package," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>
- [35] "Writing a simple publisher and subscriber (C++)," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html>
- [36] "Managing dependencies with rosdep," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Rosdep.html>
- [37] "URDF," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>
- [38] "Building a visual robot model from scratch," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html>
- [39] "Using Xacro to clean up your code," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-Xacro-to-Clean-Up-a-URDF-File.html>
- [40] "Adding physical and collision properties," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.html>
- [41] "Building a movable robot model," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-a-Movable-Robot-Model-with-URDF.html>
- [42] "Tf2," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html>
- [43] "Using URDF with robot_state_publisher (Python)," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-URDF-with-Robot-State-Publisher-py.html>
- [44] "Introducing tf2," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html>
- [45] "RViz User Guide," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html>
- [46] "Parameters," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic/About-Parameters.html>
- [47] "Understanding parameters," *ROS 2 Documentation: Humble*. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Parameters/Understanding-ROS2-Parameters.html>